REPORT 06 - XP Plan

Levi Nelson, Carlos W. Mercado

Brigham Young University-Idaho

CSE272 - Software Lifecycle Models

Instructor: Br. Edward Bullen

Date: 02/15/2025

1. OVERVIEW

1.1 Project Overview

Bank Associate Nation Keeper, Inc (B.A.N.K., Inc) has hired our company for building a mobile application that will help homeowners stay on top of their home maintenance. The mobile application includes features as follows:

- Monitors electricity usage,
- Monitor appliance upkeep,
- Monitor yard maintenance,
- Track periodic cleaning.

The application will provide a monthly checklist of home maintenance activities, allow the homeowner to check off items when they are complete, and produce a report at the end of the month. Due to the Agile methodology's emphasis on working software (Beck et al. 2001), and the XP practices of continuous integration and small releases, (Juric 2000), our software will be developed through several short integrations, each concluding with a release and retrospective. Our project will likely consist of four 3-week iterations, resulting in a total time of approximately 12 weeks.

1.2 Project Specifics

- We are building a mobile application.
- Business requirements have been included in the contract.
- Pre-contract meetings took place with customer representatives.

1.3 Implemented Software Development Methodology

1.3.1 XP Programming: Overview

In the development of this application, our company will implement the software development methodology analyzed by Radmila Juric in his paper "Extreme Programming and its Development Practices" (2000), commonly known as "XP" or "Extreme Programming".

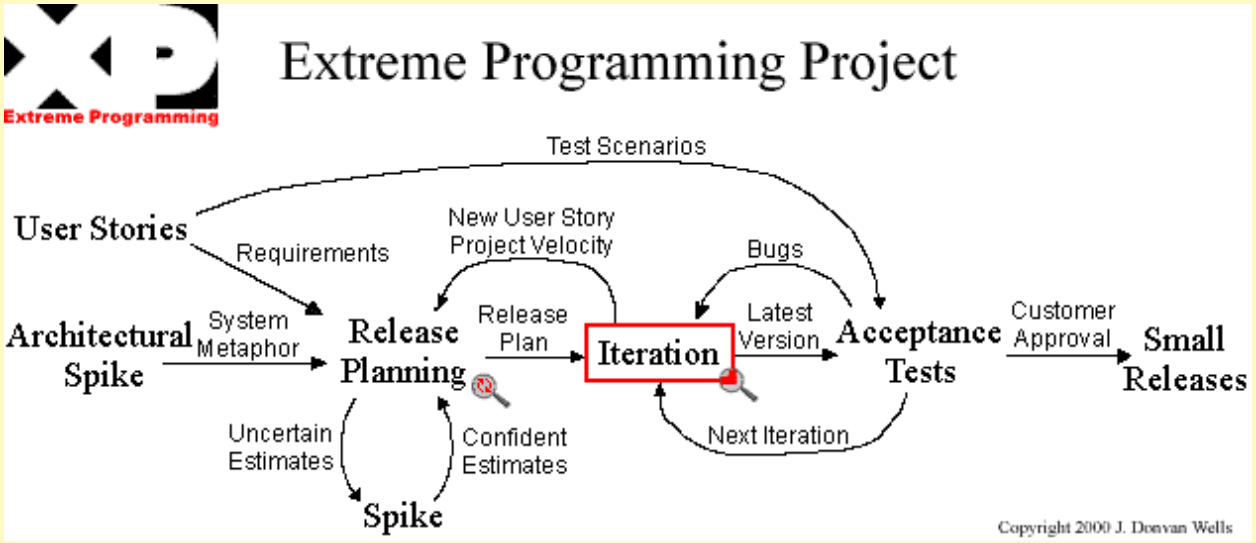

Figure 1 *- Extreme Programming's rules working together (Wells, 2013).*

This methodology is described as follows (Juric, 2000, p. 98):

1. Lightweight methodology which is efficient, low-risk, flexible, predictable, scientific and distinguishable from any other methodology.
2. As a discipline of software development practices: it strictly prescribes activities you need to complete if you want to claim that you apply XP.
3. Designed for smaller projects and teams of two to ten programmers, which result in efficient testing and running of given solutions in a fraction of a day.
4. It possesses an incremental planning approach and concrete and continuing feedback from short cycles of the software development.

5. There is strong emphasis on oral communications and automated tests that monitor progress of the software development.
6. It claims to offer a flexible schedule of the implementation of the system's functionality that actively supports changeable business needs.

Also, this software development methodology uses a series of features that enable their functioning. These features are described in the following sections: **1.3.2** through **1.3.7**. In this report we will only mention them, for an in depth description, refer to Juric (2000, p. 98-99).

1.3.2 XP Programming: Values

- "XP uses *values* as the main criteria for a successful software solution. In other words, the XP *values* serve as a guarantee, which shows that an XP's set of practices is taking the right direction."
- XP Values are: (1) Communications, (2) Simplicity, (3) Feedback, and (4) Courage.

1.3.3 XP Programming: Principles

- "The XP values are distilled into concrete *principles*, which determine XP *practices*."
- XP Principles are: (1) Rapid feedback, (2) Assume simplicity, (3) Incremental change, (4) Embracing change, and (5) Quality work.

1.3.4 XP Programming: Activities

- "The four XP *values* and those derived *principles* are a basis for building a discipline of software development *practices*. However, before *practices* are identified, XP gives a list of *activities*, which are derived from the XP *principles*."
- XP Activities are: (1) Coding, (2) Testing, (3) Listening, and (4) Design.

1.3.5 XP Programming: Practices

- "The main XP job is to structure the activities in the light of the long list of XP *practices* briefly listed below (no practice stands well on its own. It requires other practices to keep it in balance)."
- XP Practices are: (1) Planning game, (2) Small releases, (3) Metaphor, (4) Simple design, (5) Testing, (6) Refactoring, (7) Pair programming, (8) Collective ownership, (9) Continuous integration, (10) 40-hour week, (11) On-site customer, and (12) Coding standards.

1.3.6 XP Programming: Strategies

- "The implementation of XP should be based on the strategies of the XP project management where almost all XP *values*, *principles* and *practices* are employed in order to deal with a particular strategy."
- XP Strategies are: (1) Management Strategy, (2) Development Strategy, (3) Design Strategy.

1.3.7 XP Programming: Lifecycle Phases

- "XP strategies are put into practice through a lifecycle of an "ideal" XP project. It consists of a short initial development phase followed by a long-term simultaneous production support and refinement."
- XP Lifecycle Phases are: (1) Exploration, (2) Planning, (3) Iterations to First Release, (4) Productionizing, (5) Maintenance, and (6) Death.

Keywords: software development methodologies, XP, Extreme Programming

2. MEETINGS

2.1 Introductory Meeting

2.1.1 ***Who***

- *All employees, B.A.N.K. representatives Rick and Rachel.*

2.1.2 ***Agenda***

- *Welcome*
- *Overview and detailed explanation of the software to be developed.*
- *Role assignment and explanation.*
- *Question and answer session.*

2.1.3 ***Purpose***

- *Kick-off the project and build excitement for the task at hand! Assign employees to teams, appoint team leads, and allow employees the opportunity to meet with B.A.N.K. representatives.*

2.1.4 ***Event***

- *Held once at the beginning of the project.*

2.2 CRC Session

2.2.1 ***Who***

- *All employees*

2.2.2 ***Agenda***

- *Welcome*
- *CRC Card Creation*

2.2.3 ***Purpose***

- *The purpose of a CRC session is to create Class, Responsibilities, and Collaboration cards. These cards contain information and a preliminary design for objects and their functionalities within a program (UMBC, n.d). CRC cards are special, because they can be created by anyone - after all,* **"The more people who can help design the system the greater the number of good ideas incorporated" (Wells, 1999).**

2.2.4 ***Event***

- *This meeting will occur at the beginning of the project. It may need to be repeated as new design ideas are discussed.*

2.3 Daily Stand-Up

2.3.1 ***Who***

- *All employees*

2.3.2 ***Agenda***

- *Welcome*
- *Report of yesterday's tasks.*
- *Overview and selection of daily tasks.*
- *Definition and explanation of problems.*
- *Question and answer session.*

2.3.3 ***Purpose***

- **The purpose of a daily stand-up meeting is to "communicate problems [and] solutions" while also removing the necessity for other time-consuming meetings (Wells 1999). In each of these meetings, employees will report what**

tasks were accomplished the day prior, what tasks will be completed today, and what issues are causing delays (Wells 1999).

2.3.4 ***Event***

- *Recurring, held once daily each morning.*

2.4 Pair-Programming Sessions

2.4.1 ***Who***

- *Development team, in respective pairs.*

2.4.2 ***Agenda***

- *Completion of daily tasks.*

2.4.3 ***Purpose***

- **It is a major requirement of the XP process for <u>all</u> development and programming to be completed in pairs. In fact, no code can be developed without two developers present (Juric, 2000).** *This practice benefits our team by promoting cooperation and reducing the risk of simple errors in the initial development of code. During these pair-programming sessions, all daily tasks will be completed, and all code will be written.*

2.4.4 ***Event***

- *Recurring, held every day of development.*

2.5 Release Planning Meeting

2.5.1 ***Who***

- *Project Manager, On-Site Customers, Testing and Development Team Leads, Technical Writer*

2.5.2 ***Agenda***

- *Welcome*
- *User story selection*
- *Release Plan document created*

2.5.3 ***Purpose***

- **A release planning meeting must be held to create the release plan document, which outlines the entire project. This document will subsequently be used to create iteration plans. User stories will be selected for implementation, and divided into project iterations. The release plan document must be approved by the client representatives, and negotiation must occur until both parties are satisfied (Wells 1999).**

2.5.4 ***Event***

- *Once, at the beginning of the project. However, secondary meetings with the On-Site Customers may be required for further negotiation.*

2.6 Iteration Planning Meeting

2.6.1. ***Who***

- *Project Manager, On-Site Customers, Testing and Development Team Leads, Technical Writer*

2.6.2. ***Agenda***

- *Welcome*
- *User story and failed acceptance test selection*
- *Developer task selection*
- *Iteration Plan document created*

2.6.3. ***Purpose***

- *The iteration planning meeting will assist to create the iteration plan document. In this meeting,* **user stories and failed acceptance tests to be completed during the current iteration are selected. Developers will choose which tasks they would like to complete and give their own reasonable time estimates (Wells 1999).**

2.6.4 ***Event***

- *Weekly during the planning phase of the release.*

2.7 Retrospective Meeting

2.7.1. ***Who***

- *All employees, including On-Site Customers*

2.7.2. ***Agenda***

- *Welcome*
- *Collection of team feedback*
- *Areas of improvement recognized*
- *Necessary changes addressed*

2.7.3. ***Purpose***

- *The purpose of a retrospective meeting is to reflect on the previous iteration and create solutions for flaws and areas of improvement (Tripani, 2023). This will be done through collection of team feedback, allowing all team members the opportunity to share opinions on the previous sprint. Areas of improvement will then be addressed, and solutions will be planned.*

2.7.4 ***Event***

- *Recurring, to be held at the conclusion of each iteration.*

3. DOCUMENTS

3.1 CRC (Class, Responsibilities, and Collaboration) Cards

3.1.1 **Author**

- Anyone can create a CRC card. However, they will mainly be created by the Testing and Development Team Leads, as well as the Project Manager.

3.1.2 **Intended Audience**

- Development and Testing team members, Project Manager, On-Site Customers.

3.1.3 **Purpose**

- **CRC cards are used to design the system as a team. Individual cards are used to represent objects, and they help the team to simulate the system before creation (Wells 1999).** CRC cards allow for simple, easily readable system design and a more loosely defined documentation strategy.

3.1.4 **Deadline**

- *Recurring, these documents will be created during each CRC session*.

3.2 User Stories

3.2.1 **Author**

- On-Site Customers

3.2.2 **Intended Audience**

- Development and Testing team members, Project Manager, On-Site Customers.

3.2.3 **Purpose**

- **User stories are written by the customer, with the assistance of the developers, prior to the release planning meeting. A selection of user stories will be selected during the release planning meeting to be created during the**

next iteration (Wells 1999). User stories do not contain much detail, but they are essential for customer requirements to be accurately communicated to the developers for creation.

3.2.4 **Deadline**

- *Recurring, prior to each release planning meeting.*

3.3 Release Plan

3.3.1 **Author**

- Technical Writer, Project Manager, On-Site Customers, Development and Testing Team Leads

3.3.2 **Intended Audience**

- Development and Testing team members

3.3.3 **Purpose**

- **The release plan is created during the release planning meeting. It specifies which user stories are going to be implemented for each system release and defines dates for those releases (Wells 1999).**

3.3.4 **Deadline**

- *Recurring, this document will be created during each release planning meeting.*

3.4 Iteration Plan

3.4.1 **Author**

- Technical Writer, Project Manager, On-Site Customers, Development and Testing Team Leads

3.4.2 **Intended Audience**

- Development team members, Project Manager, On-Site Customers

3.4.3 **Purpose**

- **The iteration plan is created during the iteration planning meeting. User stories for this specific iteration are chosen by the on-site customers, in order of value. Failed acceptance tests to be reattempted are also included in this document (Wells 1999).** *Because failed acceptance tests are included in the Iteration Plan, the Testing Team Lead was specified as an author.* **Based on this document, developer pairs will choose tasks to complete and give their own time estimates (Wells 1999).**

3.4.4 **Deadline**

- *Recurring, this document will be created during each iteration planning meeting.*

3.5 Test Plan

3.5.1 ***Author***

- *Technical Writer, Testing Team Lead, Project Manager*

3.5.2 ***Intended Audience***

- *Testing team members, Development team members, Project Manager, On-Site Customers*

3.5.3 ***Purpose***

- *Unlike other methodologies, The XP Methodology does not require a specific testing document. However, for the sake of organization, all tests created for this project will be organized into a test plan document, which will be updated daily as new tests are run, completed, and created.* **In the XP methodology, unit tests are written before the production code is created and run every time the**

production code is edited (Juric, 2000). *This document will serve as a guide for all tests created in this project.*

3.5.4 **Deadline**

- *This document will be created at the beginning of each iteration. It will also be updated daily as tests are run and new tests are created.*

4. ROLES

4.1 Project Manager **(1 person)**

4.1.1 **Who**

- Oliver

4.1.2 **Qualifications**

- Significant experience in development, testing, and management.

4.1.3 **Responsibilities**

- The project manager should manage both the development of the product and client relations. *They should also assist in drafting documentation and designing the finished product. The Project Manager will preside over the entire development process and aim the project towards completion (Ovcharenko, 2024).*

4.2 UX Designers **(2 people)**

4.2.1 **Who**

- Ursula and Xavier.

4.2.2 **Qualifications**

- College degree in or related to graphic design or visual media. Experience in the design industry.

4.2.3 **Responsibilities**

- The UX Designers will handle all design work for the entire project. They will create wireframes and mockups using design principles. They will also ensure the end-user portions of the product are visually pleasing. *In other words, they are responsible for the visual design of the software (Ovcharenko, 2024).*

4.3 Technical Writer **(1 person)**

4.3.1 **Who**

- Teri.

4.3.2 **Qualifications**

- College degree in or related to Technical Writing, Communications, or English.

4.3.3 **Responsibilities**

- The Technical Writer will create all of the text in the product. They are responsible for, alongside the coding team, creating and proofreading all text that will be displayed to the user. The Technical Writer will also proofread code comments for accuracy. Most importantly, the Technical Writer will draft, generate, edit, and proofread all documentation and required documents. They are responsible for high-quality, readable documentation.

4.4 Development Team Lead **(1 person)**

4.4.1 **Who**

- Abe

4.4.2 **Qualifications**

- Significant experience in software engineering and project development.

4.4.3 **Responsibilities**

- The Development Team Lead is responsible for overseeing the coding team and managing product development. They will designate tasks for team members and lead team meetings. Additionally, they will work with the Technical Writer to create required documents for the project.

4.5 Development Team Members **(6 people)**

4.5.1 **Who**

- Britney paired with Larry, Claire paired with Ingrid, Grace paired with Jack

4.5.2 **Qualifications**

- College degree in software engineering, computer science, or a related field.

4.5.3 **Responsibilities**

- The Coding Team Member is responsible for developing the code for the project. They will collaborate with one another to create a working product. **Additionally, as per the XP model, they will work in pairs (Juric 2000).**

4.6 Testing Team Lead **(1 person)**

4.6.1 **Who**

- Frank

4.6.2 **Qualifications**

- Significant experience in software debugging and testing.

4.6.3 **Responsibilities**

- The Testing Team Lead is responsible for overseeing the testing team and managing the debugging process. They will delegate tasks to team members and lead team meetings. Additionally, they will work with the Technical Writer to create required documents for the project.

4.6 Testing Team Member **(4 people)**

4.6.1 **Who**

- Emily, Frank, Holly, Keith

4.6.2 **Qualifications**

- College degree in software engineering, computer science, or a related field.

4.6.3 **Responsibilities**

- Testing Team Members will test and debug the code of the product, as developed by the coding team. They will collaborate with themselves and the Testing Team Lead to fully test the code for security flaws and bugs.

4.7 On-Site Customers **(2 people)**

4.7.1 **Who**

- Rick, Rachel

4.7.2 **Qualifications**

- Employed representatives of the client.

4.7.3 **Responsibilities**

- **On-Site customers are required for the XP Programming Methodology.** Though Rick and Rachel are not employed by the software development company, **they must be available at all times for rapid feedback in unit testing and effort estimations (Juric 2000).**

5. CHECKPOINTS

5.1 Release Planning Completed

5.1.1 Effort Estimation

- *1 week after the project starts.*

5.1.2 Exit Criteria

- *A Release Planning document has been created, and to-be implemented user stories for this release have been defined. This checkpoint will only be completed upon creation of the Overview document.*

5.2 Iteration Planning Completed

5.2.1 Effort Estimation

- *1 week after iteration starts.*

5.2.2 Exit Criteria

- *A plan for the subsequent iteration has been completed, including the iteration plan document. User stories to be created for the iteration have been converted into tasks, which have been selected by developer pairs.*

5.3 Development Completed

5.3.1 Effort Estimation

- *Recurring - but at most, this will take 2 weeks per iteration.*

5.3.2 Exit Criteria

- **All tasks as defined in the iteration plan document have been completed. In other words, this iteration's development work is finished.**

5.4 Testing Completed

5.4.1 Effort Estimation

- *This checkpoint will be progressing during the development checkpoint. As such, it will end at the same time as the development checkpoint - 3 weeks at maximum.*

5.4.2 Exit Criteria

- **All defined test cases, as defined in the test plan document, are successful and free from errors. The code is clean, readable, and deployable. Additionally, the code must fulfill all safety and security requirements.**

5.5 Iteration Release

5.5.1 Effort Estimation

- *At maximum, 2 weeks after iteration starts.*

5.5.2 Exit Criteria

- *With all development and testing complete, the software for this iteration is released. This checkpoint will only be fulfilled upon successful release of this iteration's software.*

5.6 Retrospective Completed

5.6.1 Effort Estimation

- *This checkpoint will be reached at the conclusion of the iteration - at most, 2 weeks from iteration start.*

5.6.2 Exit Criteria

- *The retrospective meeting has been completed, and all team members have reviewed the iteration's successes and failures. This checkpoint will only be fulfilled upon completion of the retrospective meeting.*

5.7. Timeline

Event	Date
Release Planning Complete Deliverable: Release Plan Document	1 week after the project starts.
Iteration Planning Complete Deliverable: Iteration Plan Document	1 week after iteration starts.
Development and Testing Completed	3 weeks after iteration starts.
Iteration Released Deliverable: One Iteration's Completed, Working Software	3 weeks after iteration starts.
Iteration Retrospective Complete	3 weeks after iteration starts.

6. ANALYSIS

In the following section, we will list the strengths and weaknesses of the XP Extreme Programming Software Development Methodology. In the last part of it we will address a final summary and reflection paragraph.

6.1 Strengths/Pros

We state that the strengths this methodology has aligned with those stated by Gavriluk (2024):

1. **Robust software.** Continuous testing and test-driven development result in clean code that works now.
2. **Fast development.** Thanks to the fast pace of incremental change, continuous integration, and deployment, extreme programming lasts several months, whereas the usual approach would require years.
3. **Low cost of change.** Rapid feedback and rapid change cost less than major updates.
4. **Error avoidance.** Pair programming ensures fewer mistakes, fewer rewrites, and stable program units.
5. **Lean product.** Thanks to the YAGNI (*You Aren't Gonna Need It*) and DRY (*Don't Repeat Yourself*) rules as well as the system metaphor technique, the software is easy to maintain and enhance. The code is clear and readable all the time.
6. **Desired Product.** XP is tailored to the customer's requirements. That is why the end product meets all the demands.
7. **Team's well-being.** The vital premise for the XP work is that the team has to be motivated and rested. That is why the work is limited to a 40-hour week.

6.2 Weaknesses/Cons

Gavriluk (2024) describes the following as weakness of the XP methodology:

1. **Lots of effort.** XP requires lots of persistence, creativity, and lean thinking to adjust the system to the client's needs day by day.
2. **Customers must participate.** According to the XP framework, at least one client representative must work with the team permanently. Very often, customers have no interest and time to do so.
3. **Relatively high costs.** Pair programming means double salary and can be problematic for smaller teams with a limited budget. XP teams necessarily include a tracker and a coach, which increases expenses.
4. **Time for meetings.** In extreme programming, meetings occur very often. There are daily stand-ups that last up to 15 minutes, longer end-of-the-week meetings, the Planning Game at the beginning of each development phase, etc. Communicating face to face rather than writing messages or documentation is advisable. Altogether meetings require additional time and nerve.
5. **Location restriction.** XP projects imply that the customer participates in the development regularly and meets the team face-to-face. This is hard to implement when the customer has a distant location.
6. **Stress.** There is a lot of stress and pressure due to tight deadlines when the code has to be designed and tested today, not tomorrow. Indeed, this is why the programming is called 'extreme.'

Additionally, other authors explain the following weaknesses:

7. **Software architecture issue.** "Do highly communicative XP environments move the management control of the overall software production towards the programmer's level? (...) could generate more obstacles and as long as the XP principles do not address the

issue of software architecture more explicitly, this will remain one of the XP project's biggest weaknesses." (Juric, 2000, p. 97)

6.3 Summary & Reflection

Overall, we think that this methodology is suitable to our needs. We count on a balanced team willing to learn that will adapt to some of the challenges that this methodology proposes (like pair programming, tight deadlines, and continuous meetings). We are confident in our team's ability to accurately follow these practices.

One of the most challenging aspects of the XP progress is the continuous customer feedback loop. But thanks to the help of representatives Rick and Rachel, we are able to succeed in this aspect. Having on-site customers present at all times will assist in communication with stakeholders and B.A.N.K. The XP Methodology will allow for a working, feasible product within a small amount of time and satisfy the needs of the client.

7. REFERENCES

1. Beck, et al. (2001) Manifesto for Agile Software Development. Available at: <https://agilemanifesto.org/>. (Accessed: 14 February 2025).
2. Gavriluk, V. (2024) Back to blog / Pros and Cons of Extreme Programming, Pros and Cons of Extreme Programming. Available at: <https://arounda.agency/blog/pros-and-cons-of-extreme-programming>. (Accessed: 13 February 2025).
3. Juric, R. (2000) Extreme programming and its development practices ITI2000. Proceedings of the 22nd International Conference on Information Technology Interfaces (Cat. No.00EX411), Pula, Croatia, 2000, pp. 97-104. Available at: <https://www.researchgate.net/publication/3893585_Extreme_programming_and_its_development_practices>. (Accessed: 13 February 2025).

4. Trapani, K. (2023) How to hold effective retrospectives without wasting time, Cprime. Available at: <https://www.cprime.com/resources/blog/how-to-hold-effective-retrospectives-without-wasting-time/#:~:text=Retrospective%20meetings%20are%20a%20key,Sprint%20better%20than%20the%20last>. (Accessed: 14 February 2025).
5. University of Maryland, Baltimore County (UMBC), CRC Cards (n.d.) UMBC. Available at: <https://userpages.umbc.edu/~cseaman/ifsm636/lect1108.pdf> (Accessed 14 February 2025).
6. Wells, D. (1999) Extreme programming: A gentle introduction., Extreme Programming: A Gentle Introduction. Available at: <http://www.extremeprogramming.org/>. (Accessed: 13 February 2025).